

Rev80

A Python desktop application for capturing, analyzing, and recording vibration data from industrial rotating equipment. Designed for predictive maintenance workflows using IEPE accelerometers and voltage-output sensors connected via USB oscilloscopes (PicoScope 4000A series) or USB audio interfaces (Digiducer legacy).

Rev80 by Rev Engineering, LLC — solves 80% of your vibration needs for 10% of the cost.

Table of Contents

- [Features](#)
 - [CLI Reference](#)
 - [Installing on Windows](#)
 - [Installing from Source \(any OS\)](#)
 - [Contributing](#)
 - [Building the Windows Installer](#)
 - [Architecture](#)
 - [Module Reference](#)
 - [Data Pipeline](#)
 - [AcquisitionSettings](#)
 - [VibeSample and ChannelResult](#)
 - [ScopeSensor](#)
 - [Signal Generator](#)
 - [Simulated Sensor](#)
 - [Data Storage](#)
 - [Configuration and Persistence](#)
 - [PicoScope Integration](#)
 - [Testing](#)
-

Features

- Real-time time-domain and frequency-domain plots across multiple simultaneous channels
- Welch-based power spectral density with configurable window (Hann, Blackman-Harris, Flattop, Hamming, etc.)
- Velocity and displacement spectra derived from acceleration via frequency-domain integration
- Single-shot and continuous streaming capture modes
- Per-channel 4th-order Butterworth highpass and lowpass filters
- Configurable IEPE sensor library: sensitivity (mV/EU), modality, engineering units
- PicoScope 4000A built-in signal generator for excitation testing
- HDF5 file save/load for post-processing and archiving (v4 format)
- Configurable frame cache depth (default 32 frames) with backward browse
- Trend plot: overall vibration amplitude over time per channel
- Simulated sensor (bearing-defect signal generator) for offline development and CI testing
- Configurable amplitude modes: RMS, 0-P, P-P
- Configurable units: acceleration (g, mm/s², in/s²), velocity (mm/s, in/s, mil/s), displacement (mm, in, mil)

- **Monitor Mode** — interval datalogger: captures frames at a configurable interval (5 s – 2 days) into a single session HDF5; anomaly-triggered burst capture with three independent detection modes (EWMA-RMS broadband, EWMA-Spectral frequency-shape, and fixed-level upper/lower thresholds); configurable post-burst cooldown gate; manual “Record Burst” button
- **Session browser** — load and browse historical monitor sessions; burst events displayed as vertical markers on the vibration trend

CLI Reference

GUI mode

```
rev80                                # launch GUI
rev80 --from-file path/to/file.h5    # load measurement on startup
rev80 --from-file path/to/session_dir/ # load monitor session on startup
rev80 --init-config                  # seed config files and exit
rev80 --debug                         # verbose logging
```

`--from-file` accepts a v4 single-measurement `.h5` file or a v5 monitor session directory (containing `session.h5`). The GUI opens, displays the data immediately, and the session browser and file browser remain fully functional.

Headless mode

```
rev80-headless [options]
# or equivalently:
python -m rev80 --headless [options]
```

Before first use on a new machine, seed the config directory:

```
rev80-headless --init-config
```

This writes `acquisition.yaml` and `devices/picoscope-defaults.yaml` to `~/.config/rev80/` so you can edit them before connecting hardware.

Info commands (return immediately, no hardware or heavy imports):

Command	Description
<code>--init-config</code>	Create default config files and list them
<code>--list-devices</code>	Enumerate connected PicoScope devices
<code>--list-sensors</code>	Show the IEPE sensor library
<code>--edit-config</code>	Open <code>acquisition.yaml</code> in <code>\$EDITOR</code>

Session options:

Option	Default	Description
<code>--interval SECS</code>	600	Capture interval in seconds
<code>--pre-buffer SECS</code>	30	Pre-burst buffer duration
<code>--burst-duration SECS</code>	120	Burst capture duration

Option	Default	Description
<code>--output DIR</code>	DEVDATA/monitor/	Output root directory
<code>--no-compress</code>	—	Disable gzip compression
<code>--start-now</code>	—	Skip the pre-start confirmation prompt

Acquisition options:

Option	Default	Description
<code>--device SERIAL</code>	auto-detect	PicoScope serial number, or <code>sim</code>
<code>--channels N [N ...]</code>	from device config	Channel indices to enable (persisted to device config)
<code>--maxfreq HZ</code>	from <code>acquisition.yaml</code>	Max analysis frequency override
<code>--binsize HZ</code>	from <code>acquisition.yaml</code>	Frequency resolution override
<code>--debug</code>	—	Verbose logging to stderr

First-run device config: On the first run with a new PicoScope, headless generates `devices/picoscope-<model>-<SN>.yaml` from the defaults template before the confirmation prompt. Edit it to set channel coupling, voltage range, and sensor assignments, then restart.

Channel persistence: `--channels 0 1` updates the `enabled` flag in the saved device config so the setting is sticky across restarts — you do not need to repeat the flag.

Confirmation gate: Before connecting, headless prints a session summary (device, channels, sample rate, monitor interval, anomaly config, config file paths) and waits for Enter. Use `--start-now` to bypass this for unattended use (systemd, cron, scripts).

Example workflows:

```
# First time on a new Pi – seed config, connect scope, generate device config
rev80-headless --init-config
rev80-headless --list-devices
rev80-headless                               # generates device config, shows summary

# Edit settings, then start unattended
nano ~/.config/rev80/acquisition.yaml
nano ~/.config/rev80/devices/picoscope-4424A-JY123.yaml
rev80-headless --start-now

# One-liner with explicit overrides (changes persisted to device config)
rev80-headless --channels 0 1 --interval 300 --start-now

# Simulated sensor – offline testing, no hardware
rev80-headless --device sim --interval 10 --start-now
```

Writes a v5 `session.h5` file loadable by the GUI session browser or `--from-file`. Clean shutdown on `Ctrl+C` or `SIGTERM` (suitable for systemd `Restart=on-failure`).

Installing on Windows

1. Install **PicoSDK 11.1.418** (or PicoScope 7 for Windows) from picotech.com/downloads. **Restart your computer** after installation so Windows registers the USB kernel driver.
2. Run **Rev80Setup-<version>.exe** and follow the installer. It creates a Start Menu shortcut and an uninstaller. No admin rights required.

SmartScreen warning: the installer is currently unsigned. Click More info → Run anyway to proceed.

Runtime file locations

Purpose	Location
Measurement data (.h5)	~/Documents/Rev80/data/
Log files	~/Documents/Rev80/logs/
Config / sensor library	%APPDATA%\rev80\

Installing from Source (any OS)

Works on Windows, Linux, and macOS. Requires Python 3.10+.

1. Install **PicoSDK** or **PicoScope 7** for your OS from picotech.com/downloads. **Restart your computer** after installation.
2. Get the source and install dependencies:

```
git clone <repo-url>/vibegui.git
cd vibegui

# Runtime dependencies only
pip install .

# Runtime + development tools (pytest, pyinstaller, etc.)
pip install -e ".[dev]"
```

3. Run the app:

```
# GUI (default)
python -m rev80

# GUI – open directly on a saved measurement or monitor session
python -m rev80 --from-file DEVDATA/my_run.h5
python -m rev80 --from-file DEVDATA/monitor/2026-06-02-130000/

# Headless interval datalogger (no display required)
rev80-headless --init-config          # seed config files first
rev80-headless                        # auto-detect scope, show summary
rev80-headless --device sim --interval 10 --start-now # offline test
```

```
# With debug logging to console
python -m rev80 --debug
```

Runtime dependencies: `numpy`, `scipy`, `dearpygui==2.0.0`, `h5py`, `pyyaml`, `plyer`, `picosdk`

Headless (no display) usage:

```
# Seed config on a fresh install
rev80-headless --init-config

# Auto-detect PicoScope – shows summary, waits for Enter
rev80-headless

# Fully explicit, skip prompt (suitable for scripts/systemd)
rev80-headless \
  --interval 300 \
  --pre-buffer 30 \
  --burst-duration 120 \
  --output /mnt/nas/vibration \
  --channels 0 1 \
  --maxfreq 1000 \
  --binsize 1 \
  --start-now

# Simulated sensor (no hardware)
rev80-headless --device sim --interval 10 --start-now
```

Sessions written by the headless mode are identical v5 HDF5 files and can be loaded in the GUI:

```
# Open GUI on a specific session directory
python -m rev80 --from-file /mnt/nas/vibration/2026-06-02-130000/

# Or point at the session.h5 directly
python -m rev80 --from-file /mnt/nas/vibration/2026-06-02-130000/session.h5

# Or a regular single-measurement save
python -m rev80 --from-file DEVDATA/my_measurement.h5
```

The `picosdk` package requires the PicoScope 4000A driver (`ps4000a.dll` / `libps4000a.so`) to be present on the system for hardware use. The app will start without it and show a “driver not found” notice in the device dialog — the simulated sensor is still available.

Contributing

Development environment (Linux / macOS recommended)

```
git clone <repo-url>/vibegui.git
cd vibegui

# Create and activate a virtual environment
python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate

# Install in editable mode with dev tools
pip install -e ".[dev]"
```

```
# Run the test suite (no hardware required – uses SimulatedSensor)
pytest tests/

# Launch the app against the source tree
python -m rev80
```

Project layout

src/rev80/	Python package
_paths.py	Runtime-safe path resolution (dev vs frozen)
_pico_loader.py	Windows DLL search path setup for frozen builds
logging.yaml	Logging configuration (bundled with package)
monitor/	Monitor Mode package
__init__.py	
session.py	MonitorSession dataclass
gate.py	IntervalGate scheduler
anomaly.py	AnomalyHook protocol; Rms/Spectral/FixedThreshold hooks;
Composite	
writer.py	MonitorWriterThread (daemon, writes session.h5)
controller.py	MonitorController (is_recording; hook supplied at start)
assets/	
fonts/	CommitMono Nerd Font (gitignored – add locally)
assets/	App icon source
rev80.svg	Source artwork
make_icons.sh	Regenerates rev80.ico via Inkscape + ImageMagick
rev80.ico	Multi-resolution icon used by installer and exe
drivers/	PicoScope DLLs (Windows build only, not committed)
installer/	Inno Setup script
tests/	pytest suite
test_monitor_controller.py	
test_monitor_gate.py	
test_monitor_index.py	
test_monitor_session_load.py	full write→load→browse integration tests
rev80.spec	PyInstaller build spec
build.sh	Full build pipeline (run from Git Bash)

Replacing the app icon

Drop a new `assets/rev80.ico` in place and rebuild — no changes to `rev80.spec` or `installer/rev80.iss` are needed. To regenerate the `.ico` from the SVG source:

```
# Requires inkscape and imagemagick
cd assets
./make_icons.sh
```

Building the Windows Installer

Produces a self-contained one-directory executable and a standalone installer (`.exe`) via PyInstaller and Inno Setup. **The build must run on a 64-bit Windows machine.**

Prerequisites

Tool	Where to get it	Notes
Python 3.10+ (64-bit)	python.org	Must be 64-bit; add to PATH
Git for Windows	git-scm.com	Provides Git Bash for <code>build.sh</code>
PicoSDK 11.1.418	picotech.com/downloads	Reboot after install
Inno Setup 6	jrsoftware.org/isinfo.php	Per-user install to %LOCALAPPDATA% is fine

Install project dependencies (from Git Bash or cmd.exe):

```
pip install -e ".[dev]"
```

Running the build

```
# Full pipeline: collect DLLs → PyInstaller → Inno Setup
./build.sh

# Individual steps
./build.sh dlls           # collect PicoScope DLLs into drivers/ only
./build.sh pyinstaller    # PyInstaller only (skips DLL collection)
./build.sh installer      # Inno Setup only (requires dist/ to exist)
```

Output

Path	Description
<code>dist/rev80/rev80.exe</code>	Standalone executable (no install needed)
<code>installer/Output/Rev80Setup-<version>.exe</code>	Installer with Start Menu shortcut and uninstaller

Known constraints

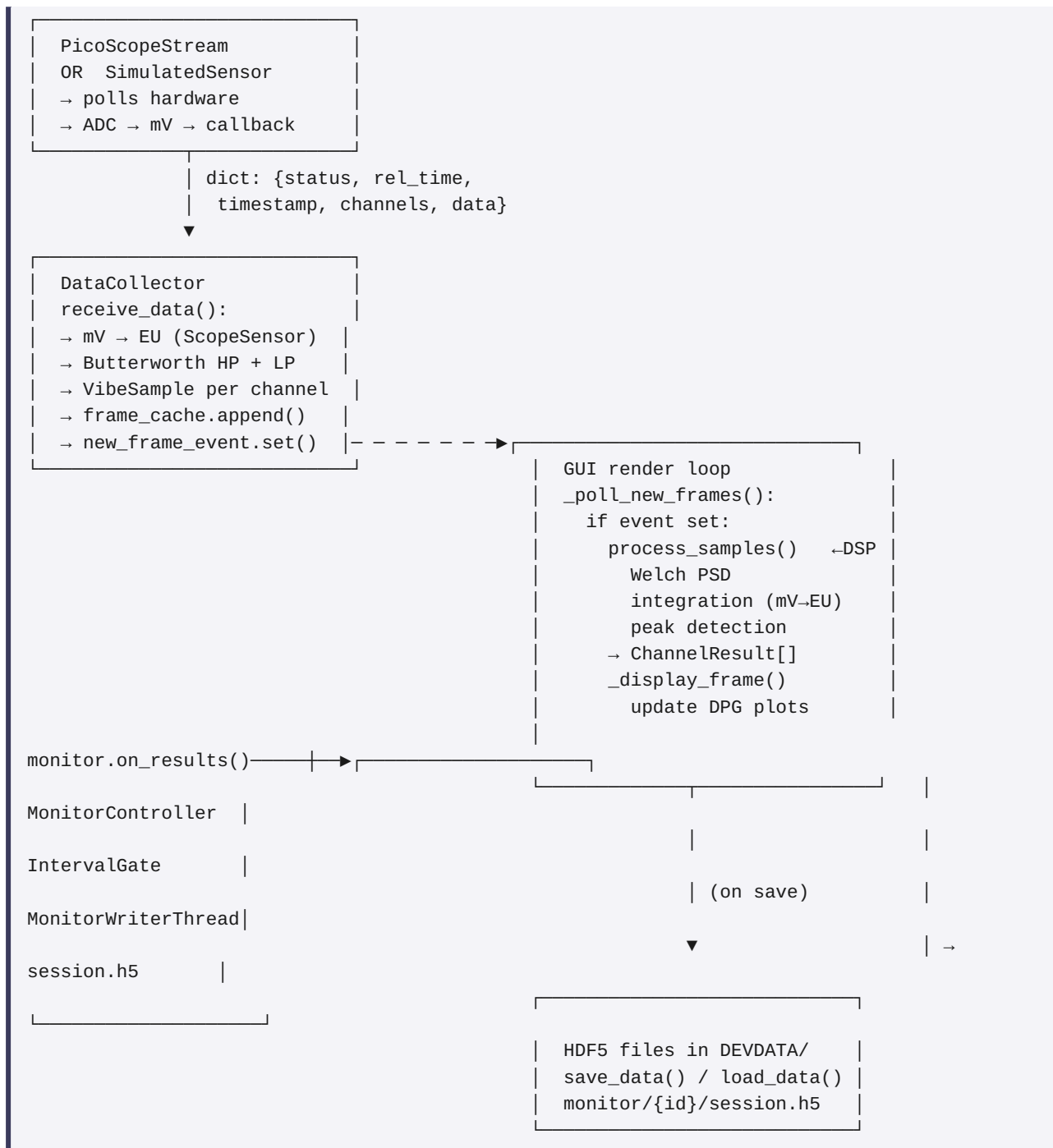
- **64-bit only** — PicoSDK DLLs are 64-bit; 32-bit Python will not work.
- **DearPyGui pinned to 2.0.0** — versions above 2.0.0 have a known viewport crash on Windows.
- **USB kernel driver** — `ps4000a.dll` is the user-mode library; the USB kernel driver is installed separately by PicoSDK. Reboot required before first hardware connection.
- **Code signing** — the installer and executable are unsigned; Windows SmartScreen will warn on first run. See `README` code signing notes or sign with `osslsigncode` and a certificate.

Architecture

The pipeline is strictly layered. The hardware thread and the GUI render loop are decoupled via a `threading.Event` — the collector never imports or calls into DPG. All DSP (Welch FFT, filtering, integration) lives in `collector.py`; `gui.py` is a pure presentation layer that consumes pre-computed `ChannelResult` objects.

Hardware thread

Main (GUI) thread



Decoupling properties: - `collector.py`, `monitor/`, `sample.py`, `picoscope.py` — zero DPG imports. Any event loop can drive them. - `new_frame_event` is a `threading.Event` — a `stdlib` primitive with no GUI dependency. - `MonitorWriterThread` runs as a daemon thread independent of both the hardware thread and GUI. - A headless process can replace the GUI by polling `new_frame_event`, calling `process_samples()`, and passing results to `MonitorController` — approximately 50 lines.

When the GUI is slower than the hardware data rate it skips to the latest frame — earlier frames remain in the ring cache (default 32 frames) for browsing. The hardware thread is never blocked by rendering.

Module Reference

Module	Responsibility
<code>__main__.py</code>	Entry point — logging setup, <code>GUI</code> instantiation, main loop, cleanup
<code>logger.py</code>	YAML-configured logging (<code>logging.yaml</code>); writes to <code>Log/</code> ; global exception hook
<code>util.py</code>	Constants (<code>MAXFREQ_PRESETS</code> , <code>BINSIZE_PRESETS</code> , <code>UNITS</code> , <code>AMPLITUDE_MODES</code> , <code>MONITOR_INTERVAL_PRESETS</code>), unit taxonomy and SI conversion, integration order helpers, <code>UI_Elements</code> DPG tag registry
<code>icons.py</code>	<code>CommitMono</code> Nerd Font (Codicons) registry; <code>load()</code> registers font with DPG; <code>IC</code> dict maps icon names to <code>\uXXXX</code> codepoints
<code>sensor.py</code>	<code>VibeSensor</code> dataclass — device metadata; <code>find()</code> enumerates hardware <code>PicoScopes</code> only; <code>simulated()</code> returns a test sensor; <code>connect()</code> returns the appropriate stream
<code>picoscope.py</code>	<code>FindPicoScope()</code> — enumerates PS4000A units; <code>PicoScopeStream</code> — polling thread, ADC→mV, overflow detection, watchdog recovery, signal generator setup
<code>scope_sensor.py</code>	<code>ScopeSensor</code> dataclass — IEPE sensor metadata: name, sensitivity (mV/EU), engineering units, amplitude mode, UUID
<code>scope_sensor_registry.py</code>	<code>ScopeSensorRegistry</code> — YAML-backed CRUD for user sensor library and per-channel assignments
<code>sample.py</code>	<code>AcquisitionSettings</code> — spectrum, filter, and cache config with derived properties; <code>VibeSample</code> — single-channel time-domain block; <code>ChannelResult</code> — frozen display-ready result from <code>process_sample()</code>
<code>config.py</code>	OS-aware config directory; per-device YAML persistence (channels, acquisition settings, monitor defaults); atomic writes; fallback to built-in defaults
<code>collector.py</code>	<code>DataCollector</code> — multi-channel acquisition state machine: stream lifecycle, per-channel Butterworth filtering, mV→EU conversion, frame ring cache, <code>new_frame_event</code> signal, DSP via <code>process_sample()</code> / <code>process_samples()</code> , trend accumulation, HDF5 save/load, monitor session loaders
<code>simulation.py</code>	<code>SimulatedSensor</code> (daemon thread) + signal generators: <code>GenerateTone</code> , <code>GenerateNoise</code> , <code>GenerateBearingVibration_SpectralMethod</code> , <code>GenerateBearingVibration_TemporalMethod</code>

Module	Responsibility
<code>gui.py</code>	<code>GUI</code> class — <code>dearpygui</code> three-column layout with manual render loop (<code>_poll_new_frames</code>), all config dialogs, spectrum/time/trend plots, file I/O, monitor card, session browser
<code>monitor/__init__.py</code>	Re-exports: <code>MonitorController</code> , <code>MonitorSession</code>
<code>monitor/session.py</code>	<code>MonitorSession</code> frozen dataclass — session parameters, config snapshots, and cooldown settings
<code>monitor/gate.py</code>	<code>IntervalGate</code> — snap-to-grid capture scheduler; burst mode entry/exit; caller-supplied time (unit-testable)
<code>monitor/anomaly.py</code>	<code>AnomalyHook</code> Protocol; <code>NullAnomalyHook</code> ; <code>RmsThresholdHook</code> ; <code>SpectralThresholdHook</code> ; <code>FixedThresholdHook</code> ; <code>CompositeAnomalyHook</code> ; <code>AnomalyEvent</code> dataclass
<code>monitor/writer.py</code>	<code>MonitorWriterThread</code> — daemon thread; appends interval frames to <code>/monitor/</code> and burst frames to <code>/burst/</code> in <code>session.h5</code> ; disk-space guard
<code>monitor/controller.py</code>	<code>MonitorController</code> — owns gate, writer, anomaly hook; anomaly hook supplied at <code>start()</code> and active for the full session; <code>trigger_burst()</code> for manual burst

Data Pipeline

1. Acquisition — `PicoScopeStream` (`picoscope.py`)

`PicoScopeStream` is a background polling thread that wraps `ps4000aRunStreaming` . On each poll:

1. Converts ADC counts → mV via `adc2mV()` for all enabled channels
2. Detects ADC overflow per channel via the overflow bitmask
3. Accumulates mV samples in a `(blocksize × N_channels)` buffer
4. When the buffer fills, fires the registered callback with:

```
{
    'status':      str,           # 'OKAY', 'OVERFLOW', etc.
    'overflow_mask': int,        # bitmask, one bit per channel
    'rel_time':     float,       # seconds since stream start
    'timestamp':    datetime,
    'unit':         ['mV', ...], # one entry per channel
    'channels':     [0, 1, ...], # enabled channel indices
    'data':         ndarray,     # shape (blocksize, N_channels), mV
}
```

A watchdog thread monitors for >5 s silence and attempts up to 3 reconnect cycles automatically.

Device discovery — `VibeSensor.find()` calls `FindPicoScope()` and returns hardware-only results.

`SimulatedSensor` is excluded; use `VibeSensor.simulated()` for offline development and testing.

2. Preprocessing — `DataCollector` (`collector.py`)

`DataCollector.receive_data(frame)` runs in the hardware callback thread:

1. For each enabled channel, extracts the channel column from `frame['data']`
2. Converts mV → engineering units using `ScopeSensor.sensitivity` (if a sensor is assigned)
3. Optionally applies a **4th-order Butterworth highpass** filter (default 10 Hz cutoff) using SOS coefficients for numerical stability
4. Optionally applies a **4th-order Butterworth lowpass** filter
5. Wraps each channel's data in a `VibeSample`
6. Assembles a frame dict `{ch: VibeSample, 'overflow': mask}` and appends it to the frame ring cache (configurable depth via `AcquisitionSettings.cache_frames`, default 32)
7. Sets `new_frame_event` to signal the GUI render loop

3. Spectral Analysis — `DataCollector.process_sample()` (`collector.py`)

`DataCollector.process_sample(ch, sample)` computes and returns a `ChannelResult`:

- **Welch PSD** — `scipy.signal.welch` with configurable window function, 50% overlap, and bin size controlled by `AcquisitionSettings.binsize`
- **Frequency-domain integration** — when the assigned `ScopeSensor.engineering_units` modality differs from the target display unit, integration is applied by multiplying the spectrum by $(1j \cdot 2\pi f)^n$ where `n` is the number of integration steps (negative = integrate, positive = differentiate)
- **Peak detection** — `scipy.signal.find_peaks` sorted descending by amplitude
- **Overall amplitude** — broadband RMS/0-P/P-P computed from time-domain data

4. Visualisation — `GUI._poll_new_frames()` / `_display_frame()` (`gui.py`)

The GUI uses a manual render loop (`while dpd.is_dearpygui_running()`). Each tick, `_poll_new_frames()` checks `DataCollector.new_frame_event`. If set, it grabs the latest frame from `frame_cache` and calls `_display_frame()`:

- Calls `DataCollector.process_samples()` → list of `ChannelResult` for each enabled channel
- Updates time-domain and spectrum line series via `dpd.set_value()`
- Updates peak table and trend plot
- If multiple frames arrived since the last tick, only the newest is rendered — earlier frames remain in cache for browsing
- Axis limits are **not** automatically adjusted; press **Autoscale** to fit all axes on demand

AcquisitionSettings

`AcquisitionSettings` (`sample.py`) centralises spectrum, filter, and channel config. Use the public properties — do not write to private `_` attributes directly.

Spectrum settings

Property	Description
<code>maxfreq</code>	Upper frequency of interest (Hz) — drives <code>samplerate</code> selection

Property	Description
<code>binsize</code>	Frequency resolution of Welch FFT (Hz) — drives <code>blocksize</code> selection
<code>samplerate</code>	Derived — minimum samplerate $\geq 2 \times \text{maxfreq}$
<code>blocksize</code>	Derived — next power of 2 satisfying <code>samplerate / blocksize</code> $\leq \text{binsize}$
<code>acquisition_period</code>	Derived — <code>blocksize / samplerate</code> (seconds)
<code>n_fft_bins</code>	Derived — number of Welch output bins
<code>fft_window</code>	Welch window function: <code>'hann'</code> (default), <code>'blackmanharris'</code> , <code>'flattop'</code> , <code>'hamming'</code> , <code>'boxcar'</code> , <code>'bartlett'</code>
<code>welch_overlap</code>	Welch segment overlap fraction (default 0.5)

Filter settings

Property	Description
<code>highpass_enabled</code>	Enable 4th-order Butterworth highpass filter
<code>highpass_fc</code>	Highpass cutoff frequency (Hz)
<code>lowpass_enabled</code>	Enable 4th-order Butterworth lowpass filter
<code>lowpass_fc</code>	Lowpass cutoff frequency (Hz)

Channel settings

Property	Description
<code>enabled_channels</code>	List of active channel indices
<code>channel_voltage_ranges</code>	Dict <code>{ch: range_index}</code> — PS4000A voltage range per channel
<code>channel_couplings</code>	Dict <code>{ch: 'AC' 'DC'}</code> — input coupling per channel

Cache settings

Property	Description
<code>cache_frames</code>	Ring buffer depth in frames (default 32); configurable via GUI Acquisition tab

Helper methods: `voltage_range_for(ch)`, `coupling_for(ch)`, `name_for(ch)`, `target_unit_for(ch)`, `amplitude_mode_for(ch)`, `copy()`.

VibeSample and ChannelResult

VibeSample

`VibeSample` (`sample.py`) holds one block of time-domain samples for a single channel.

```

sample.data                # numpy.ndarray float64, shape (N,), raw mV samples
sample.samplerate          # int - Hz
sample.unit                # str - 'mV' (raw hardware unit)
sample.timestamp           # str - ISO format timestamp
sample.rel_time            # float - seconds since stream start
sample.status              # str - 'OKAY', 'OVERFLOW', etc.
sample.overall_ampl_by_integration_order # ndarray (5,) - broadband RMS for orders -2..+2

# Cached on first call; re-computed when Welch config changes:
sample.psd_mv              # numpy.ndarray - Welch PSD (mV RMS)
sample.freq_hz             # numpy.ndarray - frequency axis (Hz)

```

HDF5 save/load is handled by `DataCollector.save_data()` and `DataCollector.load_data()`, not by `VibeSample` directly.

ChannelResult

`ChannelResult` is a frozen dataclass returned by `DataCollector.process_sample(ch, sample)`. It is the canonical display-ready result for one channel at one instant.

```

result.channel            # int
result.unit               # str - display unit
result.time_data          # ndarray - time-domain signal in display units
result.time_vec           # ndarray - time axis (seconds)
result.freq               # ndarray - full Welch frequency axis (Hz)
result.spectrum           # ndarray - PSD in display units
result.peaks              # ndarray - indices into freq/spectrum arrays
result.overall            # float - broadband amplitude (RMS/0-P/P-P per config)
result.timestamp          # datetime
result.rel_time           # float
result.status             # str

```

The `freq` and `spectrum` arrays cover the **full Welch output range** (up to Nyquist). `maxfreq` governs samplerate selection and broadband energy integration but does not crop the displayed spectrum.

ScopeSensor

`ScopeSensor` (`scope_sensor.py`) describes an IEPE sensor connected to one PicoScope channel.

```

sensor.name               # str - user label (e.g., 'PCB 352C33 Ch1')
sensor.engineering_units  # str - source modality (e.g., 'g', 'mm/s')
sensor.sensitivity         # float - mV per engineering unit (e.g., 10.2 for 10.2 mV/g)
sensor.target_unit        # str | None - display unit override
sensor.amplitude_mode     # str - 'RMS', '0-P', or 'P-P'
sensor.id                 # str - UUID, used as persistent key
sensor.notes              # str - freeform

```

Sensors are managed through `ScopeSensorRegistry` and assigned to channels via the GUI Channel Config panel. When a sensor is assigned, `DataCollector` divides incoming mV data by `sensitivity` to produce engineering units before creating `VibeSample` objects.

Signal Generator

The PicoScope 4000A has a built-in arbitrary waveform generator (AWG) on its AUX output. Rev80 exposes this through the **Generate** config tab.

Setting	Description
Enabled	Enable/disable AWG output
Waveform	Sine, Square, Triangle, DC voltage, Ramp Up/Down
Frequency (Hz)	Output frequency
Amplitude (mV pk-pk)	Peak-to-peak voltage
Offset (mV)	DC offset

Timing: The signal generator runs **continuously** from stream start to stream stop. It is programmed once when `start()` is called (`_setup_siggen()` → `ps4000aSetSigGenBuiltIn` with `PS4000A_SIGGEN_NONE` trigger source = free-running). There is no per-block triggering; the AWG and the ADC acquisition run independently and simultaneously.

Signal generator settings are persisted to `~/.config/rev80/channel_assignments.yaml` under a `siggen:` key and restored automatically when the same device reconnects.

Simulated Sensor

`VibeSensor.simulated()` returns a `VibeSensor` with `is_simulation=True`. When connected, it starts a `SimulatedSensor` daemon thread that generates synthetic bearing-defect vibration data at the configured samplerate and blocksize.

`SimulatedSensor` is **excluded from** `VibeSensor.find()` — it will never appear in the hardware device list. Use `VibeSensor.simulated()` directly in tests and offline development.

Available generators in `simulation.py`:

Generator	Description
<code>GenerateTone(config, ampl, freq, phase)</code>	Pure sinusoid
<code>GenerateNoise(config, ampl)</code>	White Gaussian noise
<code>GenerateBearingVibration_SpectralMethod(config)</code>	Bearing fault signal assembled in frequency domain (exponential noise floor + running harmonics + bearing-fault sidebands)
<code>GenerateBearingVibration_TemporalMethod(config)</code>	Bearing fault signal assembled in time domain (noise + harmonics with phase variation)

The default source for `SimulatedSensor` is `GenerateBearingVibration_TemporalMethod`. The source can be overridden at construction for targeted unit testing.

Monitor Mode

Monitor Mode turns Rev80 into a continuous interval datalogger with automatic event capture.

Interval recording

When monitoring is active, `MonitorController.on_results()` is called after every frame (from the GUI render loop or a headless polling loop). An `IntervalGate` fires at the configured interval and writes the current frame to `/monitor/{N}/` in `session.h5`. Interval captures continue regardless of whether a burst is in progress.

Burst capture

Any trigger source — automatic anomaly detection or the manual **Record Burst** button — causes the controller to:

1. Snapshot `pre_buffer_s` of raw frames from the ring cache and prepend them to the burst as pre-trigger data.
2. Capture frames at full rate for `burst_duration_s` seconds, writing to `/burst/{id}/`.
3. Tag the t=0 frame (anomaly onset, not detection time) in `burst.attrs` as `trigger_timestamp / trigger_rel_time`.
4. Start the cooldown gate to block further automatic triggers for `cooldown_s` seconds.

Anomaly detection hooks

Three hook types can be used independently or combined via `CompositeAnomalyHook` (configured by the “Hook: RMS / Spectral / Both” selector):

`RmsThresholdHook` — broadband EWMA

Maintains a per-channel EWMA baseline of `ChannelResult.overall`. Triggers when:

```
|current - baseline| / baseline > rms_pct / 100
```

...for `rms_s` seconds of sustained deviation. The baseline adapts continuously during normal operation and during burst playback. A warmup period (`warmup` frames) must elapse before triggering is enabled.

Parameter	Default	Description
<code>rms_pct</code>	10.0	% deviation from EWMA baseline
<code>rms_s</code>	3.0	Seconds signal must stay above threshold (0 = first frame)
<code>rms_alpha</code>	0.97	EWMA smoothing factor (higher = slower baseline)
<code>warmup</code>	10	Frames to collect before triggers are enabled

`SpectralThresholdHook` — frequency-shape EWMA

Compares the live PSD against a per-bin EWMA baseline. Triggers when the mean spectral deviation across the monitored frequency band exceeds `spec_pct` for `spec_n` consecutive frames. Useful for detecting new harmonics or bearing-tone shifts that don't change overall amplitude significantly.

Parameter	Default	Description
<code>spec_pct</code>	50.0	% mean per-bin deviation to trigger
<code>spec_n</code>	10	Consecutive frames required
<code>spec_fmin / spec_fmax</code>	null	Restrict band (null = full spectrum)

Use **Reset Baseline** (monitor card button) to reseed the EWMA from the current frame after a process change, speed change, or restart.

FixedThresholdHook — absolute level trigger

Fires immediately (no warmup, no EWMA) when `ChannelResult.overall` crosses a fixed level. Accepts any unit supported by `UNIT_TO_SI` and converts automatically — a threshold in `in/s` works correctly against a channel reporting `mm/s`.

Upper and lower limits are independent. A channel with no sensor/EU assigned (raw `mV`) logs a one-time warning and is silently skipped.

Parameter	Description
<code>fixed_upper_enabled</code> / <code>fixed_upper_value</code> / <code>fixed_upper_unit</code>	Trigger when amplitude rises above this level
<code>fixed_lower_enabled</code> / <code>fixed_lower_value</code> / <code>fixed_lower_unit</code>	Trigger when amplitude falls below this level

Cooldown gate

After any burst fires (automatic or manual), the controller optionally blocks further automatic triggers for `cooldown_s` seconds. Interval captures are unaffected. Use this to prevent a sustained fault from generating many overlapping burst files.

Data Storage

Manual saves (v4 format)

Single-measurement saves written by **File** → **Save** or `DataCollector.save_data()`:

```
DEVDATA/YYYY-MM-DD-HHMMSS.h5
  /metadata/
    .attrs          version=4, notes
    acquisition/     AcquisitionSettings fields
    scope_sensors/   sensor library snapshot
    channels/{ch}/    per-channel config
  /frames/{i}/
    .attrs          timestamp, rel_time, samplerate, status
    {ch}/data        (blocksize,) float64 mV, gzip-compressed
  /trend/{ch}/
    rel_times, orders (M,5) integration orders matrix
```

```
collector.save_data(Path("DEVDATA/my_run.h5"))
collector.load_data(Path("DEVDATA/my_run.h5"))
```

Monitor sessions (v5 format)

Monitor Mode writes one `session.h5` per session, appending frames as the interval gate fires:

```
DEVDATA/monitor/{session_id}/session.h5
  /metadata/
    .attrs          file_version=5, session_id, start_time, interval_s
```

acquisition/	AcquisitionSettings snapshot at arm time
scope_sensors/	sensor library at arm time
channels/{ch}/	per-channel config at arm time
/monitor/{N}/	one group per interval gate firing (N=0,1,2,...)
.attrs	timestamp, rel_time, samplerate, status, overall_json, peaks_json
{ch}/data	(blocksize,) float64 mV, gzip-compressed
/burst/{burst_id}/	one group per burst event
.attrs	trigger_type, trigger_timestamp, trigger_rel_time, burst_duration_s, max_overall_json, n_frames, n_pretrigger_frames
{frame_index}/	
.attrs	timestamp, rel_time, is_pretrigger, overall_json
{ch}/data	
/burst.attrs	burst_list – JSON array of burst summaries

session_id = "YYYY-MM-DD-HHMMSS" (UTC). Loaded via the session browser or:

```
collector.load_monitor_session(session_h5)
collector.load_monitor_burst(session_h5, burst_id)
```

Configuration and Persistence

Config lives in the OS-specific config directory: - **Linux/macOS**: \$XDG_CONFIG_HOME/rev80/ (default:

~/.config/rev80/) - **Windows**: %APPDATA%\rev80\

Run `rev80 --init-config` (or `rev80-headless --init-config`) to create the directory and seed all default files. The layout is:

```
~/.config/rev80/
acquisition.yaml          # acquisition + monitor settings (instance-wide)
scope_sensors.yaml       # IEPE sensor library (shared across all devices)
devices/
  picoscope-defaults.yaml # channel template applied to new devices
  picoscope-4424A-JY123.yaml # per-device channel + siggen config
```

acquisition.yaml — instance-wide settings

Acquisition and monitor settings are shared across all scopes on this machine. Edit this file to change capture intervals, anomaly thresholds, filter settings, etc.

```
acquisition:
  maxfreq: 1000.0          # Hz – drives sample rate (samplerate = nextpow2(2 × maxfreq))
  binsize: 1.0             # Hz – drives FFT block size
  fft_window: hann
  welch_overlap: 0.5
  highpass_enabled: true
  highpass_fc: 10.0        # Hz
  lowpass_enabled: false
  lowpass_fc: 1000.0       # Hz
  trend_max_points: 5000
  cache_frames: 15

monitor:
  interval_s: 600          # seconds between interval captures
```

```

pre_burst_s: 30          # seconds of pre-trigger data saved with each burst
burst_duration_s: 120    # seconds of post-trigger burst capture
max_burst_s: 600         # maximum burst length even if anomaly keeps retriggering
output_dir: null         # null → DEVDATA/monitor/
compression: gzip
compression_level: 4
anomaly:
  enabled: true
  hook_type: rms          # rms | spectral | both
  warmup: 10             # frames before EWMA detection activates
  rms_pct: 10.0          # % deviation from EWMA baseline to trigger
  rms_s: 3.0             # seconds signal must stay above threshold before burst fires
  rms_alpha: 0.97        # EWMA smoothing (higher → slower baseline adaptation)
  spec_pct: 50.0         # % mean per-bin deviation from EWMA baseline to trigger
  spec_n: 10             # consecutive frames required for spectral trigger
  spec_alpha: 0.995      # very slow adaptation – spectral baseline changes slowly
  spec_fmin: null        # null = full spectrum; set Hz to restrict band
  spec_fmax: null
  fixed_upper_enabled: false # burst when overall amplitude rises above this level
  fixed_upper_value: 1.0
  fixed_upper_unit: in/s    # any unit in UNIT_TO_SI; converted automatically
  fixed_lower_enabled: false # burst when overall amplitude drops below this level
  fixed_lower_value: 0.05
  fixed_lower_unit: in/s
  cooldown_enabled: false   # block re-triggers for this long after a burst fires
  cooldown_s: 300.0

```

devices/picoscope-defaults.yaml — channel template

Applied to every channel when a new device is seen for the first time. Edit this before connecting a new scope to set your preferred defaults site-wide:

```

channel:
  enabled: false          # only channel 0 is enabled on new devices
  sensor_id: null
  voltage_range: 6        # PS4000A range index (6 = ±1 V)
  coupling: AC
  channel_name: null      # null → defaults to 'Ch A', 'Ch B', ...
  target_unit: null       # null → use sensor engineering units
  amplitude_mode: 0-P

sigen:
  enabled: false
  wave_type: PS4000A_SINE
  freq_hz: 1000.0
  pktopk_uv: 1000000      # 1 V pk-pk
  offset_uv: 0

```

devices/picoscope-<model>-<SN>.yaml — per-device channel config

Channel coupling, voltage range, sensor assignments, and signal generator settings for a specific scope. Generated automatically on first connection; edit to customise each channel.

```

channels:
  0:
    enabled: true
    sensor_id: <uuid>      # from scope_sensors.yaml

```

```
    voltage_range: 6          # ±1 V
    coupling: AC
    channel_name: Motor NDE
    target_unit: in/s
    amplitude_mode: 0-P
  1:
    enabled: false
    sensor_id: null
    voltage_range: 6
    coupling: AC
    channel_name: null
    target_unit: null
    amplitude_mode: 0-P
  siggen:
    enabled: false
    wave_type: PS4000A_SINE
    freq_hz: 1000.0
    pktopk_uv: 1000000
    offset_uv: 0
```

scope_sensors.yaml — global IEPE sensor library

User-defined IEPE sensors shared across all devices. Add entries here to make sensors available for assignment in the GUI Channel Config panel or headless config:

```
- id: <uuid>
  name: PCB 352C33 Ch1
  sensitivity_mv_per_eu: 10.2  # mV per engineering unit
  engineering_units: g
  target_unit: in/s           # optional display unit override
```

Managed via `ScopeSensorRegistry` — provides CRUD operations. When a sensor is assigned to a channel, `DataCollector` divides incoming mV by `sensitivity` to produce engineering units.

Logging

Logging is configured via `src/rev80/logging.yaml`. In development, log files are written to `log/`. In a frozen Windows build, logs go to `~/Documents/Rev80/logs/`.

PicoScope Integration

Rev80 targets the **PicoScope 4000A series** as its primary acquisition hardware via the `picosdk` Python bindings.

Key implementation details

Component	Role
<code>FindPicoScope()</code>	Opens each unit in sequence; reads model, serial, build date, channel count via <code>ps4000aGetUnitInfo</code> ; returns dicts compatible with <code>VibeSensor(**dev)</code>

Component	Role
PicoScopeStream	Background thread: ps4000aRunStreaming → streaming callback → ADC → mV → accumulator → app callback

Resilience features: - USB power-source fallback: if `ps4000aOpenUnit` fails, retries with `ps4000aChangePowerSource` - Up to 5 open attempts with 1–2 s delay (extended delay after `PICO_NOT_RESPONDING`) - Watchdog: if no data arrives for 5 s, attempts up to 3 reconnect cycles before raising - Overflow logging rate-limited to once per 2 s per channel to avoid log spam

Dependencies

```
pip install picosdk
```

The PicoScope 4000A driver (`ps4000a.dll` on Windows, `libps4000a.so` on Linux) must be present on the system path.

Testing

Tests use `VibeSensor.simulated()` and synthetic data — no hardware required. The full suite runs in ~50 s.

```
# Run all tests (no hardware)
pytest tests/

# Skip hardware-dependent tests explicitly
pytest tests/ -k "not hardware and not siggen"

# Single file
pytest tests/test_monitor_session_load.py

# Single test
pytest tests/test_vibechecker.py::test_save
```

Test file	Coverage
<code>test_sample.py</code>	<code>AcquisitionSettings</code> , <code>VibeSample</code> , <code>ChannelResult</code>
<code>test_vibechecker.py</code>	<code>DataCollector</code> stream lifecycle, save/load, trend
<code>test_acquisition_settings.py</code>	Derived properties, setter validation
<code>test_scope_sensor.py</code>	Sensor calibration pipeline, mV → EU scaling
<code>test_picoscope.py</code>	<code>FindPicoScope</code> enumeration logic (mocked driver)
<code>test_config.py</code>	YAML persistence, defaults, atomic writes
<code>test_monitor_gate.py</code>	<code>IntervalGate</code> snap-to-grid, burst entry/exit
<code>test_monitor_anomaly.py</code>	<code>RmsThresholdHook</code> (warmup, streak, t=0 tracking), <code>SpectralThresholdHook</code> (EWMA, band masking), <code>FixedThresholdHook</code> (unit conversion, mV skip, modality mismatch), <code>CompositeAnomalyHook</code>

Test file	Coverage
<code>test_monitor_controller.py</code>	<code>MonitorController</code> lifecycle, hook wiring, cooldown gating, HDF5 structure
<code>test_monitor_session_load.py</code>	Full write → load → browse integration: interval frames, burst frames, trend reconstruction, NaN regression guard

Hardware-specific tests skip automatically when no PicoScope is detected.